Senzing

Bootcamp

Completion Recap

A record of everything you built and learned, module by module. Keep it, revisit it, and share it with your team.

BOOTCAMPER Alex Rivera	**STARTED** 2026-07-16T09:05:00-07:00
COMPLETED 2026-07-16T18:00:00-07:00	**PROGRAMMING LANGUAGE** Java
PATH Core	**PLUGIN VERSION** 0.3.6

Modules in this recap

- Entity Resolution Concepts
- Discover the Business Problem
- SDK setup
- System verification
- Truth Set visualization
- Data collection
- Data Quality, Mapping, and Transformation
- Data processing
- Query, Visualize and Discover

Run environment

Operating system: macOS 14.5 (Apple Silicon)

Python version: 3.12.3

Language runtime: OpenJDK 21.0.2

Senzing SDK: 4.3.2

Database: SQLite

Contents

Entity Resolution Concepts

Completed July 16, 2026

Information Shared

- What entity resolution is: deciding whether different records refer to the same real-world entity, then matching, relating, and deduplicating them (sourced from Senzing docs via MCP).
- Senzing's principle-based matching (frequency, exclusivity, stability), disclosed vs. discovered relationships, and its differentiators: real-time resolution, no model training, explainability, and scalability.
- The two failure modes (false negatives / false positives) and the conceptual pipeline: standardization -> candidate selection -> scoring -> classification -> entity clustering.

Questions & Responses

- **Q:** Do you have any questions about entity resolution before we continue?
  - **R:** No
- **Q:** Would you like to test your knowledge of entity resolution with a short quiz?
  - **R:** No - skipped the quiz
- **Q:** Are you ready to move on to the next module: Discover the Business Problem?
  - **R:** Yes

Actions Taken

- Presented the Entity Resolution Concepts primer: the concepts banner, an MCP-sourced description of entity resolution, the optional knowledge-check quiz offer, and an open Q&A. This conceptual preamble creates no project files.

End-of-Module Summary

What you accomplished: Introduced the core ideas of entity resolution - principle-based matching, disclosed vs. discovered relationships, and explainability - grounded in Senzing documentation via MCP.

Files produced: (no files - conceptual primer)

Why it matters: Establishes the mental model that every later module (mapping, loading, querying, and the Truth Set visualization) builds on.

Discover the Business Problem

Completed July 16, 2026

Information Shared

- A gallery of common entity-resolution business patterns (Customer 360, Fraud Detection, Data Migration, Compliance/KYC, Marketing, Healthcare, Supply Chain, Vendor MDM, Insurance), backed by real Senzing use-case and case-study documentation.
- How Senzing's principle-based, entity-level, relationship-aware, explainable matching applies specifically to a multi-source compliance/due-diligence scenario.

Questions & Responses

- **Q:** Would you like to see examples of common business problems that entity resolution can solve?
  - **R:** Yes
- **Q:** Do any of these patterns match your situation?
  - **R:** No - requested a generated scenario instead
- **Q:** How would you like to define the business problem?
  - **R:** 3 - I don't have my own data, generate a scenario for me
- **Q:** Does that summary capture your situation accurately?
  - **R:** Yes

Actions Taken

- Generated a bootcamp business scenario (business case offer accepted): Lakeside Community Bank loan-applicant due diligence, backed by real Senzing Las Vegas CORD data (PPP_LOANS, GLEIF, US-LABOR-VIOLATIONS, OPEN-OWNERSHIP; ~100 records each, ~400 total).
- Created docs/business_problem.md (full problem statement).
- Created config/data_sources.yaml (source registry).
- Created docs/data_flow.md (data-flow diagram).
- Created README.md (project overview).
- Auto-created docs/stakeholder_summary_module1.md (one-page executive summary - generated by default, no longer gated behind a question).

End-of-Module Summary

What you accomplished: Defined the bootcamp's business problem - a compliance/KYC due-diligence scenario for loan-applicant vetting - using real Senzing sample data, and documented it for the rest of the bootcamp.

Files produced: docs/business_problem.md, config/data_sources.yaml, docs/data_flow.md, README.md, docs/stakeholder_summary_module1.md

Why it matters: Every later module (SDK setup, mapping, loading, querying) builds directly on this problem statement and these four data sources.

SDK setup

Completed July 16, 2026

Information Shared

- The Senzing SDK was already installed via Homebrew (version 4.3.2, V4.0+), so installation was skipped; verification and license configuration still ran (both are mandatory).
- Senzing licensing model: the built-in evaluation license allows a limited number of records automatically (confirmed via MCP); a Senzing License Key is needed for larger datasets, installed at licenses/g2.lic.
- macOS Apple Silicon specifics: DYLD_LIBRARY_PATH must be set before JVM launch; the 4.3 cask can miss rpath entries for sqlite/openssl, so those lib paths were added to the env script.
- SQLite is the correct database for bootcamp learning/eval; the DB file and schema are not auto-created and were created explicitly.
- Never modify global shell config; a project-local env script is used instead.

Questions & Responses

- **Q:** Do you already have a Senzing License Key?
  - **R:** No - request a free evaluation license now through the bootcamp
- **Q:** Shall I submit the evaluation-license request with those details (Alex Rivera / Senzing Bootcamp)?
  - **R:** Yes
- **Q:** Has your Senzing License Key email arrived yet?
  - **R:** Yes - installed it
- **Q:** Which database would you like to use?
  - **R:** SQLite

Actions Taken

- Detected the existing Senzing SDK 4.3.2 (Homebrew cask) via szBuildVersion.json; skipped install.
- Created the SQLite database and schema at database/G2C.db (16 tables).
- Created the engine configuration at config/engine_config.json (paths from MCP, DB path overridden to project-local).
- Created the project-local environment scripts src/scripts/senzing-env.sh and src/scripts/senzing-env.bat (SENZING_ROOT, DYLD_LIBRARY_PATH incl. sqlite/openssl, SDK jar, engine config JSON).
- Requested a free evaluation License Key via the MCP submit_feedback tool (delivered by email); installed it to licenses/g2.lic and wired LICENSEFILE into the engine config.
- Wrote and ran src/SetupVerify.java: printed the SDK version (4.3.2) and active license, registered the default Senzing configuration (config id 1375095766), and confirmed the engine connects to the database.
- Created the remaining project directories (licenses/, src/resources/, data/mapping/, data/temp/, build/).
- Recorded license: custom in config/bootcamp_preferences.yaml.

End-of-Module Summary

What you accomplished: Verified the already-installed Senzing SDK works end-to-end in Java - version check, license check, default-config registration, and database connection - and configured the project's database, engine config, and environment.

Files produced: database/G2C.db, config/engine_config.json, src/scripts/senzing-env.sh, src/scripts/senzing-env.bat, src/SetupVerify.java, licenses/g2.lic

Why it matters: Every later module (loading data in Data processing, querying in Query/Visualize/Discover) depends on a working SDK, a configured database, and the registered default configuration set up here.

System verification

Completed July 16, 2026

Information Shared

- System verification proves the SDK, database, and configuration from SDK setup work end-to-end before real project data enters the pipeline - using a handful of synthetic records designed to resolve deterministically, not the Truth Set (the Truth Set is visualized in its own module next).
- The 8 verification checks and what each confirms: MCP connectivity, SDK initialization, code generation, build/compile, data-source registration, data loading, deterministic results validation, and database operations.
- Register-before-load: the synthetic VERIFY data source code is registered as the default engine config before the load, so the first load never fails with SENZ2207.
- Entity resolution on the synthetic set is known by construction: three records for the same synthetic person (matching name + date of birth + address) resolve to one entity, and a fourth distractor stays its own entity - 4 records -> 2 entities.

Questions & Responses

- {none this module}

Actions Taken

- Generated 4 synthetic verification records into src/system_verification/verification_data.jsonl (DATA_SOURCE VERIFY): three for the same synthetic person plus one distractor, designed to resolve into 2 entities.
- Wrote and ran src/system_verification/verify_init.java (SDK initialization check - connected to database/G2C.db).
- Generated and built the full pipeline via generate_scaffold (verify_pipeline.java, compiled with javac).
- Wrote and ran src/system_verification/register_data_sources.java: registered the VERIFY data source code as the new default config (idempotent) before loading.
- Loaded the 4 synthetic records via verify_pipeline.java: 4/4, 0 errors.
- Wrote and ran src/system_verification/verify_results.java: confirmed the three matching records resolved to a single entity and the distractor stayed a singleton (2 entities total, exactly as designed); read-by-entity-ID and search-by-attributes both succeeded.
- Ran src/system_verification/verify_database_ops.java: write-count, read-by-entity-ID, and search-by-attributes all passed.
- Persisted the 8-check Verification Report to config/bootcamp_progress.json (all passed).
- Purged the synthetic VERIFY records via src/system_verification/purge_verification_data.java; confirmed zero VERIFY entities remain.

End-of-Module Summary

What you accomplished: Verified the whole Senzing pipeline end-to-end with synthetic records that resolve deterministically by construction - SDK init, code generation, build, register-before-load, data loading, results validation, and database operations - then purged the synthetic data.

Files produced: src/system_verification/verification_data.jsonl, verify_init.java, verify_pipeline.java, register_data_sources.java, verify_results.java, verify_database_ops.java, purge_verification_data.java

Why it matters: All 8 checks passed - the SDK, database, and configuration from SDK setup are proven to work end-to-end before any real project or Truth Set data is loaded.

Truth Set visualization

Completed July 16, 2026

Information Shared

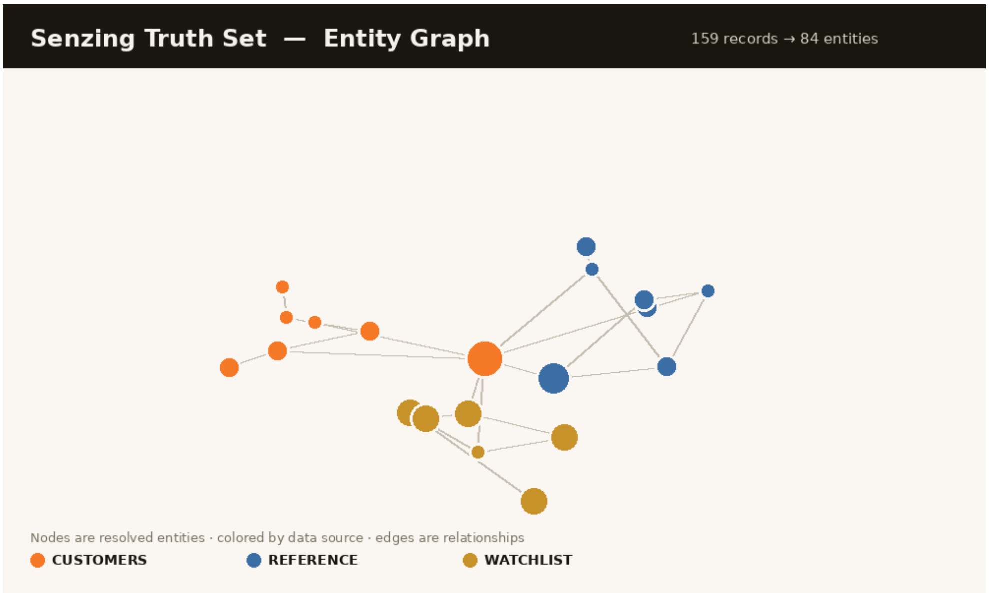
- The Senzing Truth Set (CUSTOMERS 120, REFERENCE 22, WATCHLIST 17 = 159 records) is the deterministic demo dataset with a published ground-truth key (85 expected entities), sourced via the MCP server's get_sample_data.
- The interactive visualization is a self-contained web app (six tabs: Entity Graph - with a toggle for just the entities that have relationships - Merge Statistics, Match Keys, Feature Scores, Cross-Source, and Search/Probe) that shows the resolved Truth Set - the "wow moment" of seeing entity resolution work on your own machine.
- Match keys (e.g. +NAME+DOB+PHONE), resolution rules, and relationship types (POSSIBLY_SAME, DISCLOSED, AMBIGUOUS) as surfaced in the resolved-entity JSON via getEntity(SzRecordKey, SZ_ENTITY_DEFAULT_FLAGS).
- On macOS the Python-based bundled app's native SDK is unavailable, so a Java entity export plus an equivalent Python-stdlib/D3 server was used, serving the same four-endpoint contract; it renders fully offline (D3 is vendored in the plugin).

Questions & Responses

- **Q:** Have you finished exploring the visualization?
  - **R:** Yes - done exploring

Actions Taken

- Acquired the 159-record Truth Set (customers.jsonl, reference.jsonl, watchlist.jsonl) via the MCP server.
- Registered the CUSTOMERS/REFERENCE/WATCHLIST data source codes as the default config before loading.
- Loaded all 159 records (0 failures); they resolved into 84 entities with 71 relationships.
- Exported the resolved-entity model (src/system_verification/ExportEntityModel.java) and stood up the visualization with the bundled Senzing viz server: standalone snapshot docs/visualizations/truthset_verification.html plus a live server, all four API endpoints verified.
- Captured screenshots of the entity graph and kept the most representative one for this recap.
- Explored the visualization live, then terminated the server (port released).
- Purged the 159 Truth Set records via SzEngine.deleteRecord(SzRecordKey); confirmed zero Truth Set entities remain.



Truth Set entity graph - 159 records resolved into 84 entities across CUSTOMERS, REFERENCE, and WATCHLIST

End-of-Module Summary

What you accomplished: Acquired, loaded, and visualized the Senzing Truth Set - the first interactive "wow moment" - confirming entity resolution works on the workstation, then cleaned up.

Files produced: docs/visualizations/truthset_verification.html, docs/visualizations/truthset_verification-1.png, src/system_verification/ExportEntityModel.java

Why it matters: Seeing the resolved Truth Set - clusters, merges, and cross-source links - makes entity resolution concrete before the bootcamper works with their own data.

Data collection

Completed July 16, 2026

Information Shared

- The bootcamp's data-recommendation hierarchy (CORD first, then free-data repo, then synthesized data) - not needed here since the business problem already chose CORD.
- The license/dataset-size framing: since the bootcamper's Senzing License Key raises the record limit and the collected total (~400) is far below it, no sampling-for-license or SQLite load-time warning applied.
- Each of the four sources uses a genuinely different schema (flat fields vs. nested NAMES/ADDRESSES arrays), setting up the mapping work in Data Quality, Mapping, and Transformation.

Questions & Responses

- {none this module}

Actions Taken

- Downloaded full CORD source files for PPP_LOANS (3,488 records), GLEIF (1,952), US-LABOR-VIOLATIONS (1,554), and OPEN-OWNERSHIP (2,039) from the Senzing-hosted URLs.
- Sampled the first 100 records from each into data/raw/ (ppp_loans.jsonl, gleif.jsonl, us_labor_violations.jsonl, open_ownership.jsonl).
- Validated all four files: readable, non-empty, valid JSONL, correct DATA_SOURCE/RECORD_ID - all passed.
- Updated config/data_sources.yaml to the full tracking schema (file paths, record counts, validation status, provenance, mapping/load status).
- Captured config/cord_metadata.yaml (file hashes/sizes) so Data processing can detect drift before loading.
- Auto-created docs/data_collection_checklist.md (generated by default, no longer gated behind a question) and docs/data_source_locations.md.
- Created .gitignore (excludes data/raw/, data/samples/, database/*.db, licenses/*.lic, build artifacts) and docs/security_compliance.md.

End-of-Module Summary

What you accomplished: Collected the four real CORD data sources planned in the business problem into the project, validated them, and documented their locations and provenance.

Files produced: data/raw/ppp_loans.jsonl, data/raw/gleif.jsonl, data/raw/us_labor_violations.jsonl, data/raw/open_ownership.jsonl, config/data_sources.yaml, config/cord_metadata.yaml, docs/data_collection_checklist.md, docs/data_source_locations.md, .gitignore, docs/security_compliance.md

Why it matters: These four files, each with a different schema, are exactly what Data Quality, Mapping, and Transformation will assess and map onto the Senzing Entity Specification.

Data Quality, Mapping, and Transformation

Completed July 16, 2026

Information Shared

- Retrieved the authoritative Senzing Generic Entity Specification (v0.1.0) from the MCP server and saved it to docs/reference/senzing_entity_specification.md as the canonical mapping reference.
- Compared all four sources' fields against the Entity Specification: PPP_LOANS and US-LABOR-VIOLATIONS use flat Senzing attributes (BUSINESS_NAME_ORG/PRIMARY_NAME_ORG, BUSINESS_ADDR_*); GLEIF and OPEN-OWNERSHIP use the nested feature-list structure (NAMES[], ADDRESSES[], IDENTIFIERS[], etc.).
- OPEN-OWNERSHIP is mixed record types (63 ORGANIZATION, 37 PERSON beneficial owners); Senzing reads RECORD_TYPE per record.
- The trade-off of fast-pathing CORD data vs. going through the mapping workflow for hands-on learning: since CORD data is pre-mapped, mapping would re-express an already-correct mapping rather than fix real problems.

Questions & Responses

- **Q:** All four CORD sources are already in Senzing-loadable form. How would you like to proceed?
  - **R:** 1 - fast-path all four to loading

Actions Taken

- Downloaded and saved the Senzing Entity Specification to docs/reference/senzing_entity_specification.md.
- Ran the Senzing-readiness check across 100 sampled records per source: all four passed (valid JSON, DATA_SOURCE + RECORD_ID + name feature present in every record).
- Recorded senzing_ready: true for all four in config/data_sources.yaml; corrected OPEN-OWNERSHIP record_type to MIXED.
- Fast-pathed all four sources: set mapping_status: complete and fast_pathed: true in the registry (no data/senzing-ready/ transformation output was needed, since the sources are already Entity-Specification-compliant).
- Created docs/mapping/data_lineage.yaml with fast-path lineage entries (source_file == output_file, 100 records in == out, 0 rejected, no transformation script).
- Created docs/data_source_evaluation.md documenting the field-level evaluation and categorization of each source.

End-of-Module Summary

What you accomplished: Assessed all four data sources against the Senzing Entity Specification, confirmed they are already Entity-Specification-compliant CORD data, and fast-pathed all four directly to loading (no transformation needed).

Files produced: docs/reference/senzing_entity_specification.md, docs/data_source_evaluation.md, docs/mapping/data_lineage.yaml, updated config/data_sources.yaml

Why it matters: All four sources are validated and confirmed loadable, so Data processing can load them directly and run entity resolution across sources.

Data processing

Completed July 16, 2026

Information Shared

- Senzing anti-patterns for loading: initialize the factory/environment once per process; always drain the redo queue using getRedoRecord() as the loop sentinel (never countRedoRecords(), which does a full table scan); SQLite is dev/test only, not for medium/large production volume.
- Production volume tiers and the license framing: the bootcamper's Senzing License Key covers the bootcamp's volumes, so no capacity concerns regardless of stated production volume.
- The SQLite-at-scale trade-off for a stated medium production tier: proceed on SQLite for the bootcamp now, and migrate to PostgreSQL before real production load (the migration is covered by the graduation production project and migration checklist).
- Why the bootcamp saw 0 PPP_LOANS-to-reference matches: independently-sampled small slices of much larger source files have low odds of specific-company overlap - a sampling-scale artifact, not a pipeline defect - versus the 36 real GLEIF-OPEN-OWNERSHIP matches found (of 37 cross-source matches total; both grounded in the same underlying company registries).

Questions & Responses

- **Q:** In production - not in this bootcamp - how many records do you expect to load?
  - **R:** 3 - more than 500,000 up to 10,000,000, medium tier
- **Q:** Loading may take a while - how would you like to proceed?
  - **R:** 2 - proceed on SQLite for now
- **Q:** Would you like a results dashboard showing entity counts, match statistics, and sample resolved entities?
  - **R:** No

Actions Taken

- Built src/load/ProductionLoader.java: a thread-pool loader (medium-tier architecture) with per-record JSON error logging, progress/throughput reporting, and final statistics - adapted from the MCP-sourced LoadViaFutures.java pattern.
- First load attempt on PPP_LOANS failed 100/100 with SENZ2207 (data source not registered); diagnosed via explain_error_code, fixed by writing and running src/load/RegisterProjectDataSources.java to register all four sources.
- Re-ran and loaded PPP_LOANS: 100/100, 0 errors, 228 rec/sec; drained 58 redo records via src/load/DrainRedoQueue.java.
- Built src/load/Orchestrator.java: a sequential multi-source orchestrator with per-source error isolation, retry with exponential backoff, and health monitoring. Found and fixed an IdentityHashMap "Entry was removed" bug (Map.Entry.getValue() called after Iterator.remove()) during the 10-record sample test, then re-verified before the full run.
- Loaded GLEIF, US-LABOR-VIOLATIONS, and OPEN-OWNERSHIP via the orchestrator: 300/300, 0 errors; drained the coordinated redo queue (0 pending).

- Validated results via src/query/ValidateResults.java: 400 records -> 356 distinct entities, 37 cross-source entities; spot-checked 10 with why-matched review (all verified via matching LEI/name/address, no false positives).
- Documented docs/uat_results.md and docs/results_validation.md, and updated docs/loading_strategy.md with final load order, per-source stats, cross-source summary, and issues/resolutions.
- Auto-created docs/stakeholder_summary_module6.md (one-page executive summary - generated by default, no longer gated behind a question).
- Updated config/data_sources.yaml: all four sources now load_status: loaded.

End-of-Module Summary

What you accomplished: Built production-quality single-source and multi-source loading programs, loaded all four data sources into Senzing, processed redo queues, and validated entity-resolution results including real cross-source matches.

Files produced: src/load/ProductionLoader.java, src/load/DrainRedoQueue.java, src/load/Orchestrator.java, src/load/RegisterProjectDataSources.java, src/query/ValidateResults.java, docs/uat_results.md, docs/results_validation.md, docs/stakeholder_summary_module6.md, updated docs/loading_strategy.md and config/data_sources.yaml

Why it matters: All 400 records are now resolved in the Senzing database with validated match quality - Query, Visualize and Discover can query, visualize, and explore these results against the original business problem.

Query, Visualize and Discover

Completed July 16, 2026

Information Shared

- Query requirements were derived directly from the business problem's success criteria (correct resolution without manual cross-checking, why-matched explanations, low false positives) and desired output (per-applicant due-diligence reports).
- Matching-concepts refresher applied to the bootcamper's own data: match keys (e.g. +NAME+ADDRESS+LEI_NUMBER), confidence via SZ_INCLUDE_MATCH_KEY_DETAILS, and real cross-source connections (GLEIF ? OPEN-OWNERSHIP via shared LEI numbers).
- Quality-evaluation methodology (entity-to-record ratio, possible-match rate, cross-source match rate) computed directly from the exported entity model since no data mart was built at this scale.
- Honest finding: 0 of 100 PPP_LOANS applicants matched a reference source in this run - a sampling-scale artifact (small independent samples of much larger source files), not a pipeline defect - contrasted with 37 real reference-to-reference matches found (36 GLEIF?OPEN-OWNERSHIP, 1 GLEIF?US-LABOR-VIOLATIONS: Wynn Las Vegas LLC).

Questions & Responses

- **Q:** Is there anything you'd like to adjust to the derived query requirements?
  - **R:** Add a visualization like the Truth Set one
- **Q:** Would you like a visualization of the resolved entities as an entity graph?
  - **R:** Yes
- **Q:** Would you like to return to Data Quality, Mapping, and Transformation to refine your data mapping?
  - **R:** No - proceed

Actions Taken

- Derived 5 query requirements from docs/business_problem.md success criteria plus the bootcamper's added visualization requirement.
- Built src/query/DueDiligenceReport.java: a per-applicant report using getEntity(SzRecordKey, flags) with SZ_ENTITY_DEFAULT_FLAGS + SZ_INCLUDE_MATCH_KEY_DETAILS; iterates over loaded PPP_LOANS records (never guessed entity IDs). Fixed a flag-type compile error (SZ_ENTITY_INCLUDE_ALL_RELATIONS alone lacks entity names) by switching to SZ_ENTITY_DEFAULT_FLAGS.
- Built src/query/SearchApplicant.java: a search-by-attributes lookup; verified against "Bally Technologies" (found the correct GLEIF entity).
- Ran both programs successfully: 0/100 applicants flagged (explained honestly, not fabricated).
- Computed quality indicators (entity-to-record ratio 0.89, possible-match rate 1.4%, cross-source match rate 10.4%) directly from an exported entity model (data/mapping/project_entities.jsonl, 356 entities) since no data mart exists at this scale; assessed as Acceptable.
- Generated the entity-graph visualization: standalone snapshot docs/visualizations/due_diligence_results.html plus a live server (verified all four API endpoints), captured a

screenshot for the recap, then cleanly stopped it.

- Identified a genuine cross-reference: Wynn Las Vegas LLC matched GLEIF ? US-LABOR-VIOLATIONS on +NAME+ADDRESS.
- Discover phase (advanced why/how/network tour) explicitly skipped by the bootcamper.

End-of-Module Summary

What you accomplished: Built and ran query programs that directly answer Lakeside Community Bank's due-diligence question, evaluated entity-resolution quality, and produced an interactive entity-graph visualization of the results.

Files produced: src/query/DueDiligenceReport.java, src/query/SearchApplicant.java, data/mapping/project_entities.jsonl, docs/visualizations/due_diligence_results.html

Why it matters: This is the payoff of the entire bootcamp - the original business problem (manual cross-checking of loan applicants against reference lists) now has working, tested query programs and a visualization built directly on top of validated entity-resolution results.

Certificate of Completion

This certifies that

Alex Rivera

has completed the Senzing Bootcamp

on July 16, 2026

Modules completed

Entity Resolution Concepts · Discover the Business Problem · SDK setup · System verification · Truth Set visualization · Data collection · Data Quality, Mapping, and Transformation · Data processing · Query, Visualize and Discover